Práctica: Plataforma de juegos con Laravel

En esta práctica vais a desarrollar una **aplicación web completa** utilizando Laravel como entorno servidor. No se trata de una práctica corta ni de un ejercicio centrado en implementar funcionalidades aisladas, sino de un proyecto progresivo cuyo objetivo es comprender cómo se **diseñan, publican y consumen servicios web reutilizables** dentro de una arquitectura realista.

La aplicación tendrá forma de **plataforma de juegos**. Desde el punto de vista del usuario, será una web donde se puede acceder a distintos juegos y consultar resultados. Sin embargo, desde el punto de vista técnico, el sistema funcionará como un **CRM**, ya que gestionará usuarios, roles, juegos, sesiones de juego y datos asociados al uso de la plataforma. Esta doble visión es intencionada y forma parte del aprendizaje: entender que una aplicación web no se define solo por lo que se ve, sino por cómo se estructura internamente.

Laravel será el núcleo del sistema. Se encargará de la autenticación, el control de accesos, la gestión de usuarios y la exposición de los servicios web. No obstante, Laravel no resolverá todas las necesidades del sistema por sí solo. Parte fundamental de esta práctica consiste en aprender **qué responsabilidades deben asumirse desde el servidor y cuáles deben delegarse en otros componentes**, como aplicaciones cliente o servicios externos.

Los juegos no estarán programados dentro de Laravel. Se desarrollarán en lenguaje cliente, utilizando threejs en 0612-Desenvoulpament Web en Entorn Client (Javi), y se comunicarán con el servidor exclusivamente mediante **servicios web**. Esto obliga a diseñar una API clara, coherente y reutilizable, desacoplada de la interfaz y pensada para ser consumida por distintos clientes.

A lo largo de la práctica trabajaréis con una arquitectura que incluye:

- una aplicación web y una API claramente separadas,
- clientes que consumen servicios web de forma real,
- integración de servicios externos mediante API,
- y publicación del sistema en un entorno distinto al local (0614-Desplegament d'Aplicacions Web)

El proyecto no se resuelve en un único paso, sino que se construye por fases. Cada fase añade funcionalidad sobre una base ya existente, de forma que el sistema vaya ganando complejidad sin perder coherencia. Este enfoque permite entender mejor cómo evolucionan los proyectos en entornos profesionales y por qué es importante tomar buenas decisiones desde el inicio.

El objetivo principal de la práctica no es únicamente que la aplicación funcione, sino que el sistema tenga **sentido técnico**, esté bien estructurado y pueda explicarse con claridad. Se valorará especialmente la capacidad para justificar las decisiones adoptadas en el diseño de los servicios web, en su consumo desde clientes y en su publicación en un entorno real.

Esta práctica está directamente alineada con los **Resultados de Aprendizaje RA6 y RA7 del módulo 0613 – Desarrollo Web en Entorno Servidor**, y servirá para demostrar que sois capaces de trabajar con servicios web reutilizables en un contexto similar al que encontraréis en el ámbito profesional.

Creación del proyecto y fundamentos técnicos iniciales

Antes de comenzar con la gestión de juegos, es imprescindible construir correctamente la **base técnica del proyecto**. Esta fase no es solo preparatoria, sino estructural: todo lo que se desarrolle más adelante dependerá de que esta base esté bien diseñada y alineada con la teoría trabajada en el módulo.

El proyecto se inicia con la creación de una nueva aplicación en **Laravel**, utilizando su versión actual. Desde el primer momento se debe trabajar siguiendo las **buenas prácticas explicadas en clase**, especialmente en lo referente a estructura del proyecto, separación de responsabilidades y uso correcto del framework. No se trata de improvisar soluciones, sino de aplicar de forma práctica los conceptos teóricos vistos sobre desarrollo en entorno servidor.

Configuración del entorno y uso de las herramientas de Laravel

Una vez creado el proyecto, se configura el entorno de trabajo mediante el archivo de variables de entorno y la conexión con la base de datos. Laravel proporciona una serie de herramientas técnicas que deben usarse correctamente desde el inicio, entre ellas:

- **Migraciones**, para definir y versionar la estructura de la base de datos.
- **Modelos Eloquent**, para representar las entidades del sistema y gestionar las relaciones entre ellas.
- **Controladores**, para separar la lógica de negocio de las rutas.
- **Middleware**, para controlar el acceso a rutas y servicios.
- **Sistema de autenticación**, para gestionar usuarios y sesiones.

Estas herramientas no son opcionales. Su uso forma parte de la evaluación, ya que demuestran que el alumnado entiende cómo Laravel abstrae la lógica del servidor y facilita el trabajo con servicios web.

Base de datos y ORM

El sistema trabajará con una base de datos relacional, preferiblemente **PostgreSQL**, ya que el proyecto gestionará usuarios, sesiones, resultados, eventos y datos que evolucionan con el tiempo.

Laravel utiliza su ORM, Eloquent, para interactuar con la base de datos. En esta fase se deben definir correctamente las entidades principales, al menos:

- usuarios,
- roles,
- relación entre usuarios y roles.

El objetivo no es tener todo el modelo final cerrado desde el principio, sino construir una base coherente que permita añadir después juegos, sesiones de juego y datos asociados sin romper lo existente.

Autenticación y gestión de usuarios

La plataforma debe contar desde el inicio con un **sistema de autenticación real y funcional**. Laravel proporciona los mecanismos necesarios para gestionar el registro, el inicio de sesión y la protección de rutas.

Todos los servicios web relevantes estarán asociados a un usuario autenticado. No se permiten accesos directos ni simulaciones, ya que el control de identidad es clave para entender cómo funcionan los servicios web en un entorno real.

Tipos de usuario y su función en el sistema

Desde esta fase inicial deben existir claramente definidos los distintos tipos de usuario, ya que condicionan todo el funcionamiento posterior de la plataforma.

El **administrador** es el usuario con mayor nivel de control. Se encarga de gestionar usuarios, asignar roles y mantener la configuración general del sistema. No participa en los juegos como jugador, sino como responsable del sistema.

El **gestor** es el usuario encargado de la plataforma de juegos. Puede crear, editar y publicar juegos, así como consultar resultados y datos de uso. No gestiona usuarios administradores.

El **jugador** es el usuario final de la plataforma. Su función es acceder a los juegos publicados, jugar y consultar sus propios resultados. No tiene acceso a la gestión interna del sistema.

Estos roles no son solo una clasificación teórica. Laravel debe utilizarlos para controlar rutas, acciones y acceso a servicios web. El control se realiza siempre en el servidor, aplicando la teoría vista sobre autorización y seguridad.

Separación entre aplicación web y API

Antes de introducir la lógica de juegos, se establece una **separación clara entre la aplicación web y la API**. Esta separación es uno de los ejes centrales de los RA6 y RA7.

La aplicación web se define mediante las rutas de ***`web.php`*** y está pensada para la interacción directa con el usuario: autenticación, panel principal, gestión interna y navegación.

La API se define en ***`api.php`*** y expone **servicios web reutilizables**, pensados para ser consumidos por distintos clientes: los juegos, servicios externos o futuras aplicaciones. Estas rutas no devuelven vistas, sino datos estructurados.

Si se elige la separación, se debe respetar estrictamente y forma parte de los criterios de evaluación.

Uso de Inertia para la capa de vistas

Para la capa de presentación, se puede utilizar el enfoque clásico de vistas Blade o trabajar con **Inertia**.

Inertia permite usar **Vue** o **React** como capa de vistas sin convertir el proyecto en una SPA independiente. Laravel sigue siendo el backend y controla las rutas, la autenticación y los permisos, mientras que Vue o React se utilizan únicamente para la interfaz.

Este enfoque es especialmente útil para el CRM, ya que permite crear interfaces más dinámicas manteniendo la estructura del proyecto en Laravel.

Relación con los juegos en cliente

Las vistas del CRM pueden desarrollarse con Inertia, Vue o React, pero los **juegos** pueden no forman parte directa de esta capa. Los juegos se desarrollarán en cliente utilizando JavaScript y **Three.js**, dentro del módulo 0612 – Desarrollo Web en Entorno Cliente.

Si elegís hacerlo por separado al Laravel, estos juegos se comunicarán con Laravel mediante la API. No acceden directamente a la base de datos ni a la lógica interna del servidor. Esta separación refuerza la idea de servicios web reutilizables y desacoplados.

Estado esperado al finalizar esta fase inicial

Al finalizar esta parte del proyecto, el sistema debe contar con:

- un proyecto Laravel correctamente instalado y configurado,
- base de datos operativa y conectada mediante ORM,
- autenticación funcional,
- usuarios con roles definidos y controlados desde el servidor,
- separación clara entre web y API,
- capa de vistas implementada con Blade o Inertia.

Con esta base técnica y teórica correctamente aplicada, el proyecto está preparado para avanzar hacia la gestión de juegos y la relación con los clientes de la API sin improvisaciones ni refactorizaciones innecesarias.

El CRM de juegos y la relación con el cliente

En este punto ya tenemos una aplicación Laravel funcionando con usuarios autenticados y roles definidos. Ahora el proyecto empieza a tener sentido funcional: vamos a convertir Laravel en una **plataforma de juegos**, no solo en un sistema de usuarios.

La idea clave de esta parte es entender que **Laravel no ejecuta juegos**, pero sí los gestiona. Eso significa que Laravel decide qué juegos existen, quién puede acceder a ellos y qué información se guarda cuando alguien juega.

Qué es un juego para Laravel

Para Laravel, un juego es un recurso más del sistema, igual que lo puede ser un usuario o un post en otra aplicación. No conoce la lógica interna del juego ni necesita conocerla.

Lo único que Laravel debe saber de un juego es:

- qué juego es
- si está disponible o no
- quién puede acceder
- dónde se encuentra
- qué datos devuelve cuando alguien juega

Por eso, el primer paso es modelar el juego como entidad dentro del CRM.

El modelo Game y su sentido

Vais a crear un modelo `Game`. No se trata de llenarlo de campos sin criterio, sino de que represente exactamente lo que el sistema necesita.

Un juego necesita información básica para poder ser gestionado:

- un título y una descripción, para mostrarlo en la plataforma
- un estado, que indique si está publicado o no
- una referencia a la ubicación del juego, que puede ser una ruta o una URL
- el usuario que lo ha creado

Con esto ya podéis hacer algo importante: publicar y despublicar juegos sin tocar código. Un gestor decide desde el panel si un juego está disponible para los jugadores.

El panel de gestión de juegos

Los usuarios con rol de administrador o gestor acceden a un panel donde pueden gestionar los juegos. Este panel forma parte del CRM y vive en las rutas definidas en `web.php`.

Desde aquí se pueden crear juegos nuevos, editar los existentes y decidir si están publicados. También se puede acceder a una vista previa del juego, para comprobar que la integración funciona.

Este panel no es complejo en apariencia, pero es clave para evaluar si entendéis bien:

- cómo se construye un CRUD en Laravel
- cómo se valida la información
- cómo se aplican permisos de forma correcta

Un jugador no debe poder acceder a este panel, aunque conozca la URL.

Cómo ve los juegos un jugador

El jugador accede a una lista de juegos disponibles. Esa lista se genera desde Laravel filtrando únicamente los juegos publicados.

Cuando el jugador selecciona un juego, Laravel carga una vista que integra ese juego en el contexto de la plataforma. Puede ser una vista Blade o una vista gestionada con Inertia si estáis usando React o Vue.

En ese momento, el usuario sigue dentro de la sesión de Laravel. El juego no es una página aislada, forma parte de la aplicación.

Dónde viven los juegos técnicamente

Aquí hay una decisión importante que debéis tomar y entender.

Los juegos pueden vivir dentro del propio proyecto Laravel como recursos estáticos, o pueden vivir fuera, como aplicaciones cliente independientes, y Laravel solo guarda la URL de acceso.

Las dos opciones son válidas. En esta fase no importa tanto cuál elijáis, sino que tengáis claro que:

- Laravel no depende del código del juego
- el juego depende de la API de Laravel para comunicarse

Más adelante, cuando hablemos de despliegue, esta decisión tendrá consecuencias prácticas.

Por qué necesitamos una API para los juegos

El juego no debe comunicarse con rutas web. No debe enviar formularios ni depender de vistas.

El juego se comunica con Laravel exclusivamente a través de la API definida en `api.php`. Esto obliga a pensar en términos de backend y cliente, incluso aunque todo esté en el mismo proyecto.

Aquí se empieza a trabajar con una idea importante: el juego es un cliente más de la API, igual que podría serlo una app móvil.

Inicio de una sesión de juego

Cuando un jugador entra en un juego, el primer paso es informar a Laravel de que una partida va a comenzar.

El juego hace una llamada a la API para iniciar una sesión. Laravel comprueba que:

- el usuario está autenticado
- el juego existe y está publicado
- el usuario tiene permiso para jugar

Si todo es correcto, Laravel crea una sesión de juego en la base de datos y devuelve un identificador de sesión al cliente.

Ese identificador se usará durante toda la partida.

Qué datos se guardan en esta fase

En esta parte del proyecto no se trata de guardar todo. Se guarda lo necesario para poder analizar el uso del juego más adelante.

Se guarda:

- quién ha jugado
- a qué juego
- cuándo ha empezado y terminado
- cuánto ha durado
- qué resultado ha obtenido

Más adelante se añadirán eventos más detallados y datos emocionales, pero ahora la prioridad es que esta base funcione bien.

Reconocimiento facial con microservicio Python en Docker

En esta parte del proyecto vamos a integrar **reconocimiento facial**. No es un añadido decorativo ni un “extra curioso”. El objetivo es aprender a **sacar fuera de Laravel** una responsabilidad pesada y especializada, y a integrarla correctamente como servicio.

La idea clave que debéis tener clara desde el principio es esta:
Laravel no hace reconocimiento facial.
Laravel coordina, valida y decide. El reconocimiento lo hace otro sistema.

Por qué el reconocimiento facial no va en Laravel

Laravel es un framework web. Está pensado para:

- gestionar usuarios y sesiones
- aplicar reglas de acceso
- exponer una API
- guardar datos

Las librerías de reconocimiento facial trabajan con modelos de visión artificial, dependencias pesadas y procesos que no encajan bien dentro del ciclo normal de una petición web de Laravel.

Por eso, en lugar de forzar a Laravel a hacer algo que no le corresponde, se crea un **microservicio independiente** en Python.

Qué papel tiene el reconocimiento facial en esta práctica

El reconocimiento facial se usa como **verificación adicional**, no como sustituto del sistema de login.

El usuario sigue teniendo:

- su cuenta
- su contraseña
- su sesión normal en Laravel

El reconocimiento facial sirve, por ejemplo, para:

- confirmar la identidad antes de acceder a determinadas funciones
- reforzar el acceso a la plataforma de juegos
- aprender a integrar biometría de forma controlada

Desde el punto de vista formativo, lo importante no es “entrar con la cara”, sino entender cómo se integra un sistema externo sin romper la arquitectura.

El enrolamiento: registrar una foto del usuario

Para poder comparar un rostro, el sistema necesita una referencia previa. Esa referencia es una imagen del usuario que se registra conscientemente.

El enrolamiento ocurre cuando el usuario **ya está autenticado** en el CRM. El usuario entra en una sección de configuración de seguridad y registra su imagen facial.

En ese momento:

- el navegador captura o sube una imagen
- Laravel recibe la imagen
- Laravel la guarda asociada al usuario

Laravel no analiza la imagen. No calcula nada. No decide nada.
Solo la guarda de forma controlada para poder usarla más adelante.

Este punto es importante: **Laravel no hace reconocimiento ni siquiera durante el enrolamiento**.

Verificación facial usando la webcam

Cuando el sistema necesita verificar la identidad del usuario, se usa la webcam del ordenador.

Desde el navegador:

- se solicita permiso para usar la cámara
- se captura una imagen actual
- esa imagen se envía a Laravel

Laravel recibe ahora dos cosas:

- la imagen registrada previamente
- la imagen actual capturada por la webcam

Aun así, Laravel no compara imágenes. Aquí entra el microservicio.

El microservicio Python y su responsabilidad

El microservicio está desarrollado en Python y se ejecuta en un contenedor de **Docker**. Vive completamente separado del proyecto Laravel.

Su responsabilidad es única y muy concreta:

- recibir dos imágenes
- compararlas
- devolver un resultado técnico

Para hacer esta comparación se utiliza una librería especializada, por ejemplo **DeepFace**, que ofrece modelos de reconocimiento facial ya entrenados.

El microservicio no sabe nada del sistema:

- no conoce usuarios
- no gestiona sesiones
- no decide accesos

Solo compara rostros y devuelve información.

Por qué Docker es obligatorio aquí

Docker no se usa por comodidad, se usa por necesidad.

Las librerías de reconocimiento facial en Python tienen:

- dependencias nativas
- versiones concretas
- requisitos que no deben mezclarse con el entorno PHP

Docker permite aislar todo eso en un contenedor que:

- se ejecuta igual en desarrollo y producción
- se puede desplegar en un servidor o una **Raspberry**
- no contamina el entorno de Laravel

Esto os obliga a pensar en términos de servicios que se comunican por red, no en términos de “ejecutar código Python desde PHP”.

Comunicación entre Laravel y el microservicio

El navegador **nunca** se comunica directamente con el microservicio Python.

El flujo siempre es el mismo:

- el navegador envía la imagen actual a Laravel
- Laravel recupera la imagen registrada del usuario
- Laravel envía ambas imágenes al microservicio
- el microservicio responde con el resultado
- Laravel interpreta la respuesta y toma la decisión

Esta estructura es fundamental para mantener el control de seguridad en un solo punto.

La decisión siempre la toma Laravel

El microservicio devuelve datos técnicos, por ejemplo:

- si los rostros coinciden
- un valor de distancia o confianza

Pero **no concede accesos**.
No inicia sesiones.
No autoriza acciones.

Laravel recibe la respuesta y aplica sus propias reglas:

- aceptar la verificación
- rechazarla
- pedir repetir el proceso
- combinarla con otros factores

Esto mantiene la lógica de seguridad centralizada y evita depender ciegamente de un sistema externo.

Detección de emociones en el cliente y registro en la plataforma

En esta parte añadimos **detección de emociones** mientras el usuario juega. Es importante entender desde el principio que esta funcionalidad **no tiene el mismo papel que el reconocimiento facial** que vimos antes.

Aquí no estamos verificando identidades ni tomando decisiones de acceso.
Aquí estamos **observando una interacción** para poder analizar cómo se usa un juego.

Por eso, la forma de implementarlo es distinta.

Dónde se hace la detección de emociones y por qué

La detección de emociones **se hace en el navegador**, no en el servidor.

Esto no es una decisión técnica sin más, es una decisión de diseño y de responsabilidad.

El navegador:

- ya tiene acceso a la webcam
- puede procesar imágenes localmente
- puede obtener resultados sin enviar vídeo al servidor

Usar una librería JavaScript como **face-api.js** permite analizar expresiones faciales directamente en el cliente.

Laravel no recibe imágenes, no recibe vídeo y no procesa caras.
Laravel solo recibe **datos ya interpretados**.

Qué tipo de datos se detectan

La detección emocional no es “leer sentimientos”. Técnicamente, lo que se obtiene son probabilidades de expresiones faciales básicas.

Por ejemplo:

- *neutral, happy, sad, angry, surprised*

La librería devuelve valores numéricos. A partir de ahí, el cliente decide qué enviar al servidor.

No se envían datos continuamente. No se trabaja a 30 frames por segundo.
Se decide un intervalo razonable, por ejemplo cada segundo o cada pocos segundos, para no generar ruido ni exceso de datos.

Qué se envía a Laravel y qué no

Este punto es clave para entender bien el proyecto.

Desde el cliente **no se envían imágenes**.
No se envían vídeos.
No se envían datos biométricos en bruto.

Lo que se envía a la API de Laravel son datos como:

- sesión de juego
- emoción dominante en ese momento
- nivel de confianza del modelo
- instante dentro de la sesión

Laravel recibe información abstracta, no datos sensibles directos.

Relación entre emociones y sesiones de juego

Las emociones no se guardan “por usuario” de forma permanente.
Se guardan **asociadas a una sesión de juego concreta**.

Esto es importante para el diseño de la base de datos y para la interpretación posterior.

Una emoción solo tiene sentido en un contexto:
- en qué juego
- en qué momento
- durante qué sesión

De esta forma, más adelante se puede analizar, por ejemplo:

- en qué momento del juego aparecen emociones negativas
- si una versión nueva del juego genera menos frustración
- cómo responde el usuario a ciertos niveles o mecánicas

Almacenamiento de estos datos

Laravel guarda estos eventos emocionales en PostgreSQL. Puede ser una base de datos local o una gestionada con **Supabase**.

Cada registro suele incluir:

- referencia a la sesión de juego
- emoción detectada
- valor de confianza
- timestamp

No se guarda identidad biométrica ni información que permita reconstruir un rostro.

Chat en tiempo real con WebSockets

En esta parte vamos a añadir **comunicación en tiempo real** a la plataforma. No se trata de crear un chat genérico, sino de integrar un sistema de mensajería que tenga sentido dentro del proyecto y que se apoye en la arquitectura que ya tenéis.

El chat sirve para acompañar la experiencia de juego y para aprender a trabajar con **eventos en tiempo real** dentro de una aplicación Laravel moderna.

Por qué un chat y por qué en tiempo real

Hasta ahora, toda la comunicación con Laravel ha sido basada en peticiones HTTP normales. Esto es suficiente para muchas cosas, pero no para situaciones donde varios usuarios deben ver información al instante.

Un chat es un buen ejemplo porque:

- varios usuarios participan al mismo tiempo
- los mensajes deben aparecer sin recargar la página
- el servidor debe notificar a los clientes

Aquí es donde entran los WebSockets.

La solución oficial en Laravel

Laravel ofrece una solución integrada para trabajar con WebSockets llamada `Laravel Reverb`.

Usar la solución oficial tiene varias ventajas:

- se integra con el sistema de eventos de Laravel
- respeta la autenticación del framework
- evita dependencias externas innecesarias
- sigue el mismo estilo que el resto del ecosistema

No se trata solo de “que funcione”, sino de aprender a trabajar con herramientas que encajan bien con Laravel.

Qué tipo de chat vamos a implementar

El chat no es global para toda la plataforma. Está **contextualizado**.

Esto significa que:

- un chat puede estar asociado a un juego
- o incluso a una sesión de juego concreta
- solo participan usuarios que tienen acceso a ese contexto

Por ejemplo, dos jugadores que están jugando al mismo juego pueden comunicarse entre ellos, pero no con usuarios que no están en ese juego.

Relación entre chat, usuarios y permisos

El chat respeta los roles y permisos del sistema.

Laravel controla:

- quién puede entrar en un canal
- quién puede enviar mensajes
- qué información se transmite

Un usuario no puede conectarse a un canal de chat simplemente conociendo su nombre. El acceso se valida usando la sesión autenticada de Laravel.

Esto refuerza una idea importante: **el tiempo real no elimina la seguridad**, simplemente cambia la forma de comunicarse.

Cómo funciona técnicamente el flujo

Cuando un usuario entra en una página de juego, el cliente se conecta a un canal de WebSocket correspondiente a ese juego o sesión.

Cuando el usuario envía un mensaje:

- el mensaje se envía al servidor
- Laravel dispara un evento
- ese evento se emite a los demás usuarios conectados al mismo canal

Los demás clientes reciben el mensaje y lo muestran inmediatamente.

Todo esto ocurre sin recargar la página.

Persistencia de los mensajes

Aquí hay una decisión de diseño que debéis tomar conscientemente.

Podéis:

- no guardar los mensajes y tratarlos como comunicación efímera
- o guardar los mensajes en base de datos asociados a un juego o sesión

Las dos opciones son válidas, pero debéis justificar la decisión.

Si se guardan mensajes, se guardan como:

- texto, autor, referencia al juego o sesión, timestamp

No se guardan datos innecesarios.

Organización del proyecto, MCP y trabajo en equipo

Hasta ahora hemos construido una base técnica sólida: usuarios, roles, juegos, sesiones, reconocimiento facial, emociones y chat. A partir de aquí el proyecto deja de ser solo “una aplicación que funciona” y pasa a ser **un proyecto mantenible**, algo que varias personas pueden desarrollar sin caos.

Esta parte es clave porque en proyectos reales el problema no suele ser escribir código, sino **coordinar cambios, mantener coherencia y evitar errores repetidos**.

El proyecto vive en GitHub

Todo el proyecto se gestiona en **GitHub**. No se trabaja con carpetas sueltas ni con versiones locales sin control.

El repositorio contiene:

- el código de Laravel
- la configuración del microservicio Python
- documentación mínima del proyecto
- instrucciones de despliegue

Cada cambio relevante pasa por Git. No se suben versiones finales “a mano”.

Trabajo en equipo y responsabilidades

Desde el punto de vista del equipo, no todo el mundo hace de todo al mismo tiempo.

En este proyecto hay perfiles claros:

- personas que trabajan más el backend
- personas que trabajan más los juegos cliente
- personas que se encargan de integrar y probar

Esto no significa compartimentos estancos, pero sí que hay responsabilidad sobre partes concretas.

Aquí entra en juego la organización del repositorio y del flujo de trabajo.

Uso del MCP oficial de Laravel

Para ayudar en esta organización se utiliza el **MCP oficial de Laravel**. No es un sustituto del trabajo humano, sino una herramienta de apoyo para mantener orden y coherencia.

El MCP se utiliza para tareas como:

- generar plantillas de issues claras
- revisar que una pull request cumpla ciertos criterios
- recordar pasos antes de integrar cambios
- mantener consistencia en el proyecto

Esto ayuda especialmente cuando varias personas trabajan sobre el mismo código base.

Issues y tareas

Cada funcionalidad nueva se plantea como una issue. No se trabaja “directamente sobre main”.

Una issue describe:

- qué se quiere hacer
- en qué parte del sistema afecta
- qué se considera terminado

Esto obliga a pensar antes de programar y facilita que otra persona entienda el cambio.

Pull requests como herramienta de aprendizaje

Los cambios se integran mediante pull requests.

Una pull request no es solo “subir código”, es una oportunidad para:

- explicar decisiones
- recibir feedback
- detectar errores antes de que entren en producción

Aquí es donde el MCP ayuda revisando estructura, coherencia y puntos básicos, pero la revisión humana sigue siendo imprescindible.

Automatización básica del proyecto

El repositorio incluye automatizaciones sencillas:

- comprobaciones básicas antes de aceptar cambios
- verificación de formato y estructura
- preparación para despliegue

No se trata de crear un sistema complejo de CI/CD, sino de entender que un proyecto real **no depende solo de la memoria del programador**.

Relación con el resto del sistema

Esta parte conecta con todo lo anterior:

- los cambios en el backend afectan a los juegos
- los cambios en la API afectan al microservicio
- los cambios en la estructura afectan al despliegue

Por eso es importante tener un flujo claro y predecible.

Despliegue real y cierre del proyecto

Llegados a este punto, la aplicación ya no es un ejercicio académico aislado. Tenéis un sistema con varias piezas que funcionan juntas: un CRM en Laravel, una API, juegos en cliente, un microservicio en Python, tiempo real y análisis de datos. Ahora toca hacer algo que muchas veces se deja fuera en clase: **ponerlo en funcionamiento fuera del entorno local**.

El despliegue no es un añadido final. Es parte del diseño. Muchas decisiones que habéis tomado antes empiezan ahora a tener consecuencias claras.

Separar qué se despliega y dónde

Lo primero que hay que entender es que **no todo se despliega en el mismo sitio ni de la misma forma**.

Laravel, que es el núcleo del sistema, necesita un entorno con PHP, servidor web y base de datos. Esa parte se comporta como una aplicación clásica de backend.

Los juegos, en cambio, son aplicaciones cliente. Una vez compilados, no necesitan PHP ni servidor dinámico. Solo necesitan ser servidos como archivos estáticos.

Y el microservicio de reconocimiento facial es otra cosa distinta: es un servicio Python que debe ejecutarse como proceso, no como web estática.

Separar mentalmente estas tres piezas es clave para no mezclar responsabilidades.

Desplegar Laravel como aplicación principal

El CRM y la API de Laravel se despliegan juntos, porque forman parte del mismo proyecto. Se pueden desplegar en un servidor Linux normal o incluso en una `Raspberry Pi` si el uso es moderado.

Aquí Laravel actúa como:

- backend del sistema
- gestor de usuarios
- API para los juegos
- punto de control de seguridad

No es necesario que este servidor sea potente, pero sí estable. Lo importante es que Laravel tenga acceso a la base de datos y pueda comunicarse con el microservicio Python por red interna.

El microservicio Python en producción

El microservicio de reconocimiento facial se despliega **como contenedor Docker**, igual que en desarrollo. No se reescribe ni se adapta “a mano”.

Esto es una de las ventajas de haberlo separado desde el inicio.

En producción:

- el contenedor se ejecuta en el mismo servidor o en otro
- Laravel se comunica con él mediante HTTP
- el servicio no es accesible directamente desde internet si no es necesario

Desde el punto de vista del sistema, este servicio es solo “otro backend”, pero con una responsabilidad muy concreta.

Dónde se despliegan los juegos

Los juegos no necesitan vivir en el mismo servidor que Laravel. De hecho, es mejor que no lo hagan.

Una vez compilados, los juegos pueden desplegarse en plataformas de hosting estático como `Vercel`, `Netlify` o `Cloudflare`.

Estas plataformas están pensadas para servir archivos de forma rápida y eficiente. Laravel solo necesita conocer la URL del juego para integrarlo en la plataforma.

Esto tiene varias ventajas prácticas:

- los juegos cargan rápido
- no sobrecargan el servidor de Laravel
- se pueden actualizar sin tocar el backend

Cómo se conectan todas las piezas en producción

Aunque estén en sitios distintos, todas las partes siguen comunicándose de la misma forma que en local.

El juego se comunica con la API de Laravel para:

- iniciar sesiones
- enviar resultados
- registrar eventos y emociones

Laravel se comunica con el microservicio Python para:

- verificar rostros cuando es necesario

Nada de esto depende de que estén en el mismo servidor físico. Lo único que importa es que las URLs y las credenciales estén bien configuradas.

Aquí se ve claramente por qué la separación entre web.php y api.php era tan importante desde el inicio.

Qué se espera que funcione al final

Al final del proyecto, el sistema debería poder usarse de forma real, aunque sea con pocos usuarios.

Un usuario puede:

- entrar en la plataforma
- autenticarse
- acceder a juegos publicados
- jugar
- comunicarse en tiempo real
- generar datos de uso y emociones

Todo esto sin estar en localhost y sin depender del entorno de desarrollo.

RÚBRICA DE EVALUACIÓN – RA6

Módulo 0613 – Desarrollo Web en Entorno Servidor

RA6: Servicios web reutilizables (diseño, publicación y consumo)

Descripción del RA6
Diseña e implementa servicios web reutilizables, los consume desde diferentes clientes e integra servicios externos mediante APIs, aplicando criterios de seguridad, validación y coherencia.

Indicador	Excelente (4)	Notable (3)	Suficiente (2)	Insuficiente (1)
RA6.1 Diseño de servicios web	Servicios web claros, bien estructurados y reutilizables. Rutas, métodos HTTP y respuestas coherentes e independientes del cliente.	Servicios bien definidos y funcionales, con margen de mejora en coherencia o reutilización.	Servicios funcionales pero muy ligados a un caso concreto o con diseño mejorable.	Servicios mal diseñados, confusos o mezclados con lógica de vista.
RA6.2 Separación web / API	Separación clara y consistente entre web.php y api.php. Arquitectura bien definida y mantenible.	Separación correcta con pequeñas incoherencias puntuales.	Separación parcial o poco clara, pero funcional.	No existe una separación clara entre web y API.
RA6.3 Consumo de servicios web desde cliente	Consumo correcto desde clientes, con validación de datos y gestión de errores.	Consumo correcto con gestión básica de errores.	Consumo funcional sin control de errores o con dependencias rígidas.	Consumo incorrecto o inexistente de los servicios web.
RA6.4 Integración de servicios web externos	Integración correcta de servicios externos (microservicio Python), segura y desacoplada.	Integración funcional con poca abstracción o documentación.	Integración básica, funcional pero con dependencias fuertes.	No integra servicios externos o lo hace incorrectamente.
RA6.5 Validación y seguridad	Validación de datos, control de accesos y protección coherente de los servicios web.	Validación y seguridad correctas con aspectos mejorables.	Validación mínima o parcial.	Sin validación ni control de acceso adecuado.

Ponderación RA6: 50 % de la práctica

RÚBRICA DE EVALUACIÓN – RA7

Módulo 0613 – Desarrollo Web en Entorno Servidor

RA7: Publicación y consumo de servicios web

Descripción del RA7
Publica servicios web funcionales en entornos reales y demuestra su consumo desde diferentes clientes, explicando el proceso de despliegue y funcionamiento del sistema.

Indicador	Excelente	Notable	Suficiente	Insuficiente
RA7.1 Publicación de servicios web	Servicios web publicados y funcionando fuera del entorno local de forma estable.	Servicios publicados con funcionamiento correcto, con pequeños ajustes pendientes.	Servicios ejecutables fuera de local, pero con limitaciones.	No se demuestra la publicación real de los servicios.
RA7.2 Consumo real de los servicios publicados	Consumo correcto de los servicios web publicados desde clientes reales.	Consumo funcional con pequeñas carencias.	Consumo básico y limitado de los servicios.	No se demuestra consumo real de los servicios publicados.
RA7.3 Funcionamiento del sistema desplegado	El sistema completo funciona desplegado (CRM, API, juegos y servicios externos).	El sistema funciona con algún componente parcialmente limitado.	Funcionamiento básico pero incompleto del sistema desplegado.	El sistema no funciona fuera del entorno local.
RA7.4 Explicación del proceso de despliegue	Explica claramente el proceso de publicación y consumo de los servicios web.	Explicación correcta pero poco detallada.	Explicación muy básica del despliegue.	No sabe explicar el proceso de despliegue.

Ponderación RA7: 50 % de la práctica

CÁLCULO DE LA CALIFICACIÓN FINAL

- RA6: 50 %
- RA7: 50 %